`/aida-review` vs `/ultrareview`

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — verify Claude Code on Web pricing against code.claude.com/docs/en/cla code-on-the-web before relying on the cost line; the free-tier wording in this doc was captured live from the launch prompt.

The TL;DR: **`/aida-review` is the cheap, integrated lifecycle layer. `/ultrareview` is the depth layer when stakes warrant. Compose them — they're not substitutes.**

Different scopes, complementary

	`/aida-review` (AIDA skill)	`/ultrareview` (Claude Code built-in)
Engine	Local single agent (the active Claude session)	Cloud-based multi-agent review
Cost	Free — already paying for the active Claude session, no per-review charge	Free tier (3 uses per the prompt observed 2026-05-12); afterwards consumes Claude Max quota or is billed depending on plan — verify against current pricing docs
Driven by	AIDA's req metadata — walks each linked SPEC-ID's acceptance criteria against the diff	The diff itself — multiple agents read the same code with explicitly different framings
Output	Per-spec verdicts (✔ / ⚠ / ✘) + consolidated PR comment + handles merge + flips status to Completed	Multi-perspective review report; doesn't touch requirement status
Best for	AIDA-integrated workflow: spec-driven review with lifecycle bookkeeping	Deep code review when you want fresh eyes on the PR diff regardless of req tracking
Launchable from	Anywhere a Claude session can run	User-triggered only (Claude Code controls) — an agent cannot launch it for you

---

What `/ultrareview` catches that `/aida-review` misses

Captured live during PR-15 review (2026-05-12): `/aida-review` approved + merged; running `/ultrareview` afterwards surfaced three real bugs. The pattern is consistent and worth naming.

Category	Concrete example from PR-15	Why a single-agent walk tends to miss it
Cross-reference inconsistency	`review_title_matches` was case-insensitive but `derive_scope_from_entry` was case-sensitive	One prompt biases toward one perspective; the walk doesn't naturally ask *"do all consumers of this string agree?"*
Format-spec edge cases	TOML inline comments (`key = "v" # cmt`) were not stripped	The acceptance walk verifies spec compliance, not *"does the parser handle everything the FORMAT spec allows?"*
Multiplicity reasoning	`--steal` handled N=1 lease but not N>1	The singular case dominates testing; the multiplicity step is a separate cognitive move that single-framing review can skip past

`/ultrareview` wins on these because the multi-agent fleet uses **explicitly different framings** — one agent is told "find bugs," another "verify acceptance," etc. The composite findings catch what any single mental model would miss.

Post-STORY-109 (2026-05-12): `/aida-review` now includes an explicit adversarial deep-pass between the spec walk and the merge gates. The phase codifies four systematic probes — cross-reference consistency, format-spec edge cases, multiplicity (0/1/N), and "looks safe but isn't" adversarial framing — that historically slip past a single "verify acceptance" walk. The three PR-15 misses (TASK-81, TASK-84, TASK-85) all fall inside the probes' catch radius. `/aida-review (post-augment)` catches edge cases that single-agent reviews historically miss; **`/ultrareview` still wins on multi-perspective depth** — the cloud fleet brings genuinely independent framings that no single agent can fully replicate. AIDA's own answer to independent framings is the autonomous drain's separate reviewer phase (a reviewer session that never shares context with the implementer it reviews — now backed by a structural self-merge guard the implementer can't bypass) and `/aida-code-review` for exhaustive quality passes: narrower than the cloud fleet, but the independent-critic principle is internalized rather than absent.

What `/aida-review` does that `/ultrareview` doesn't

It's tempting to read "`/ultrareview` catches more bugs" as "`/ultrareview` wins." It doesn't, because **the lifecycle layer is the actual value AIDA-review claims to provide**.

- **Per-spec verdicts.** `/aida-review` walks each `// trace:SPEC-ID` link in the diff and produces a verdict against that spec's acceptance criteria. `/ultrareview` doesn't know what your requirements are.
- **Status bookkeeping.** A clean `/aida-review` pass flips the linked specs to `Completed` and removes them from the queue. The PR-merged → queue-cleared loop closes without anyone touching `aida edit`.
- **Free and unlimited inside the session.** No per-review quota draw. You can run it on every PR, including small polish ones, without thinking about budget.
- **Agent-launchable.** An autonomous agent driving `/aida-pickup` can fold review into the loop. `/ultrareview` requires a human at the keyboard.

The complementary workflow

For **substantive PRs** (new public surface, complex logic, hard-to-revert changes):

1. **`/aida-review`** first — free, integrated with lifecycle, fast. Get the spec-walk verdict.
2. **`/ultrareview`** second — multi-agent depth, ~5–10 min cloud round-trip. Spend the quota draw on the change that warrants it.
3. **Address `/ultrareview` findings** via fixup commits on the same PR.
4. **Merge.** The `/aida-review` lifecycle hooks already ran; the merge button closes the loop.

For **routine PRs** (small, well-bounded, tight blast radius):

1. **`/aida-review`** alone suffices. The 3 free `/ultrareview` uses (per the prompt observed 2026-05-12) reserve nicely for the *substantive* bucket — feature PRs with new public surface or complex logic. Polish/cleanup PRs don't usually justify the spend.

Honest scope statement

`/aida-review`'s value **isn't** "better review than `/ultrareview`." It's *"review that knows about your requirement graph and handles the lifecycle bookkeeping."* That's a complementary capability, not a substitute.

If a team has to pick one of the two for budget reasons, the right answer depends entirely on what they're optimizing for:

- *"Catch the most bugs per review"* → `/ultrareview`. Multi-agent fleets dominate single-agent walks on bug density.
- *"Keep the queue and status fields honest with the code"* → `/aida-review`. No competitor in this slot.

Both fit in most workflows. The point of this doc is that picking *one* in a binary way is almost always wrong.

See also

- STORY-109 — `/aida-review` adversarial phase. Landed 2026-05-12. See step 4 of aida-review skill.
- /aida-review skill
- composition.md — generic guidance on layering AIDA with other tools (future page).